

Daily Scholar Papers Report — 2026-07-07

2026-07-07

Daily Scholar Papers Report — 2026-07-07

[Download PDF](#)

Window covered: 2026-07-06 → 2026-07-07 (Google Scholar alerts + user-curated self-emails, last 24 h)

Executive Summary

Tuesday’s inbox delivered eight Scholar-alert threads (Xin Xia, Thorsten Holz, David Lo, Qingkai Shi, Recommended articles, Zhenchang Xing, Xiao Cheng, and a duplicate co-author thread for Gelei Deng that dedups against the Zhenchang Xing HECATE thread). Ten followed-researcher candidates and seven from the Recommended batch — seventeen raw. The self-forwarded queue was empty for the window. After Stage-1 screening, one followed alert is a scope-report artefact rather than a research paper and is set aside, and six Recommended candidates are cut as saturated DL-for-vulnerability-detection or narrow smart-contract-vuln variants. That leaves **seven for deep analysis: two Outstanding, four Keep, one Borderline-High**. The preference layer — dominated by `venue::preprint` (posterior 0.75) — gives arXiv-published work a mild tailwind but does not swing any decision on its own.

Outstanding: 2 · **Keep:** 4 · **Borderline High-Priority:** 1

Highlighted Papers

#	Title	Authors	Venue	Link
1.1	AgentFlow: Building Agent Dependency Graphs for Static Analysis of Agent Programs	S. Wang, X. Hou, Y. Zhao, X. Cheng, H. Wang	arXiv 2607.01640, July 2026	arXiv
1.2	SFA-Miner: Mining Path-Sensitive API Usage Patterns via Symbolic Finite Automata	J. Jiang, M. Zheng, Q. Shi, X. Zhang	IEEE Symposium on Security and Privacy (S&P), 2026	IEEE
2.1	BOUND: Mitigating Package Hallucinations in Large Language	S. Liu, Y. Zhao, X. Hu, K. Liu, X. Yang, X. Xia	arXiv 2607.02052, July 2026	arXiv

#	Title	Authors	Venue	Link
	Models via Model Editing			
2.2	Refploit: Facilitating Exploit Construction via Code-Agent Trajectory Repair	Z. Chen, Z. Xue, J. Zhou, X. Hu, X. Xia, X. Yang	arXiv 2607.01760, July 2026	arXiv
2.3	HECATE: Rethinking Complexity Metrics for LLM-Integrated Applications: Beyond Source Code	Z. Xu, Y. Li, G. Deng, Y. Liu, Z. Xing	arXiv 2607.01903, July 2026	arXiv
2.4	On Prompt Learning for FQN Inference: Sensitivity and Usefulness Analysis	Z. Luo, Q. Huang, Z. Xing, J. Sun, Q. Lu, J. Lu	ACM TKDD, 2026	DOI
3.1	Artificial Intelligence Support for Software Architecture Practice: A Systematic Review and Future Directions	A. Bucaioni, M. Weyssow, J. He, Y. Lyu, D. Lo	ACM TOSEM, 2026	DOI

Outstanding

1.1 · STATIC-ANALYSIS-FOR-AGENTS · First static-analysis framework for LLM-agent programs; ADG surfaces 238 taint-style prompt-to-tool risks across a 5,399-program corpus 

AgentFlow: Building Agent Dependency Graphs for Static Analysis of Agent Programs S. Wang, X. Hou, Y. Zhao, X. Cheng, H. Wang — arXiv 2607.01640, July 2026 — [arXiv link](#)

Agent programs combine ordinary host-language code with framework-defined semantics for models, prompts, tools, memory, and multi-agent orchestration. Their runtime behavior therefore depends not only on classical control- and data-flow but on a new class of *agent dependencies* — framework-induced semantics such as agent constructors, tool decorators, and agent-handoff declarations. Existing AST-based static analysis and dependency-tracking tools do not recover these dependencies.

Approach. AgentFlow introduces the *Agent Dependency Graph* (ADG): a framework-agnostic representation in which agents, prompts, models, capabilities, memory states, and control policies appear as typed nodes, and their component-dependency, control-flow, and data-flow relationships appear as typed edges. On top of ADGs, the paper implements Agent Bill-of-Materials (BOM) generation and prompt-to-tool risk detection.

Evaluation. Implemented for five representative agent frameworks. Assessed on **AgentZoo**, a corpus of **5,399 real-world agent programs**. AgentFlow recovers richer agent entities and dependencies than

AST-based baselines, produces more dependency-aware Agent BOMs, and **uncovered 238 taint-style prompt-to-tool risks** in real-world agent code.

Why it matters. The ADG is the right level of abstraction for the emerging “agent supply-chain and prompt-injection surface” problem — it decouples five different framework semantics onto one graph shape, which makes it composable with existing taint and provenance analyses.

Closing line (author’s own): “These results show that ADG provides a practical foundation for understanding, governing, and securing emerging agent software.”

1.2 · SPECIFICATION-MINING · Path-sensitive API-usage-pattern miner via symbolic finite automata — IEEE S&P 2026 

SFA-Miner: Mining Path-Sensitive API Usage Patterns via Symbolic Finite Automata J. Jiang, M. Zheng, Q. Shi, X. Zhang — IEEE Symposium on Security and Privacy, 2026 — [IEEE link](#)

APIs are the joints of modern software, and API misuse is a durable source of security-relevant bugs. Classical API-usage miners — PR-Miner, Jadet, GrouMiner, and their descendants — extract sequences or subgraphs of API calls but typically abstract path conditions away, producing patterns whose “must-hold” preconditions are lost. The consequence is well-documented: false-positive-heavy specifications and rules that only fire on the paths the miner happened to see.

Approach. SFA-Miner represents each mined pattern as a *symbolic finite automaton* whose transitions carry predicates (path conditions). Mining becomes a joint problem over sequences of API calls *and* the conjoined predicates that make each transition legally reachable. Predicate refinement lets the miner distinguish “this API pair always co-occurs” from “this API pair co-occurs only when a particular guard holds,” which is exactly the information downstream misuse-detection tooling needs to reason about programs it has never seen.

Take. Path-sensitivity was the missing dimension in FSA-based specification mining. Publishing at S&P is the right signal for how much scaffolding this had to build; the paper is worth reading for the SFA construction alone. Abstract-only in this digest (paywalled venue) — full read recommended once the PDF is accessible.

Keep

2.1 · LLM-SUPPLY-CHAIN · Model-editing framework cuts package-hallucination rate by 79.9% while preserving valid recommendations 

BOUND: Mitigating Package Hallucinations in Large Language Models via Model Editing S. Liu, Y. Zhao, X. Hu, K. Liu, X. Yang, X. Xia — arXiv 2607.02052, July 2026 — [arXiv link](#)

LLMs used for library recommendation, code generation, and dependency configuration occasionally invent package names that do not exist. In a package ecosystem, that is not a benign hallucination — attackers can (and have) registered typosquatted or fully-fabricated packages under hallucinated names, turning an LLM suggestion into a supply-chain attack vector.

Framing. BOUND (the paper’s tool) reformulates mitigation as a *package-validity boundary editing* problem, where the boundary is the model’s ability to distinguish valid from hallucinated package names in a given task context.

Approach. Two steps. First, a *risk-aware localization* strategy locates the specific model modules responsible for package hallucination. Second, those modules are edited with lightweight LoRA adapters under a *boundary-aware objective* that simultaneously reinforces valid packages, suppresses hallucinated ones, and preserves the model's other behavior (locality).

Headline numbers.

- **Package-HR** ↓ **79.9%** on edit prompts, ↓ **65.4%** on unseen prompts.
- Generalization: ↓ **12.8%** Package-HR in code generation, ↓ **34.0%** in pip-install recommendation.

Take. Cleaner than retraining or RAG-plus-blocklist, and the “boundary” framing carries reasonable intuition. Open question is drift: whether the learned boundary survives future model updates or must be re-edited each release.

2.2 · EXPLOIT-AUTOMATION · Trajectory-repair framework hits 80.2% Java-exploit reproduction — a 64.3% relative gain over the underlying code agent 

Refploit: Facilitating Exploit Construction via Code-Agent Trajectory Repair Z. Chen, Z. Xue, J. Zhou, X. Hu, X. Xia, X. Yang — arXiv 2607.01760, July 2026 — [arXiv link](#)

Reproducing disclosed Java-library exploits into runnable form is chronically manual work — public exploit references are incomplete, unstructured, or only sketch reproduction steps. Recent code agents promise automation, but the paper documents a subtle failure mode: agent-produced exploits sometimes *look* successful without ever triggering the actual vulnerable logic (a common shortcut is replacing the vulnerable API with a self-implemented function that mimics its signature).

Core insight. A failed trajectory is not garbage; it may have completed real subtasks and pinpointed dead-end directions to avoid.

Pipeline. (1) *Differential execution* validates whether an agent-generated exploit actually engages the vulnerable API path. (2) When it does not, Refploit performs *trajectory-progress analysis* to identify which segments of the failed attempt encoded real progress. (3) Constraints derived from those segments guide a focused recovery attempt.

Headline numbers.

- Datasets: **three open-source Java vulnerability datasets, 172 exploit references, 143 vulnerabilities.**
- Under DeepSeek-V4-Flash: **80.2% reproduction rate.**
- **+64.3% relative** improvement over the initial trajectory of the underlying code agent.
- Beats PoCGen (prior SOTA) and Codex + GPT-5.4.
- Consistent gains when swapped onto a different code agent — the trajectory-recovery layer is agent-agnostic.

Take. Differential execution as a soundness check for agent-produced artefacts is transferable well beyond exploit reproduction — any pipeline where “the output ran, but did it run the *right* thing” matters.

2.3 · LLM-APP-QUALITY · Prompt-as-Specification lifts prompts into first-class complexity metrics — only 10 of 52 candidates survive strict size-controlled filtering 

HECATE: Rethinking Complexity Metrics for LLM-Integrated Applications: Beyond Source Code Z.

Xu, Y. Li, G. Deng, Y. Liu, Z. Xing — arXiv 2607.01903, July 2026 — [arXiv link](#)

LLM-integrated applications blend natural-language prompts with program code, and much of their runtime behavior originates in the prompt layer rather than in the code layer. Classical complexity metrics (cyclomatic complexity, Halstead, RFC, LoC) operate solely on code and therefore miss this behavioral logic entirely.

Core idea. *Prompt-as-Specification* — a Hoare-logic-inspired formalism that interprets every prompt as a specification of intended behavior. From this framing, prompt complexity is well-defined and amenable to structural analysis.

Methodology. Grounded in **25 complexity dimensions** synthesized across published taxonomies, HECATE generates **52 candidate metrics**. Each metric is evaluated on **118 components** collected from **18 open-source repositories** using maintenance activity (drawn from version history) as an empirical proxy for complexity. Crucially, any metric that loses statistical significance once code size is added as a covariate is discarded — this rules out metrics that are only re-measuring LoC.

Headline numbers.

- **Only 10 of 52 metrics survive** the size-controlled filter.
- **7 of the 10** belong to a newly introduced family the authors call *structural breadth* — counts of structurally distinct elements (LLM call sites, memory attributes, prompt templates) rather than volume.
- **3 conventional survivors:** RFC (which exhibits similar breadth character), and Halstead N and V (surviving only as residual size effects). The new metrics outperform all three.
- Prompt-layer metrics remain significant even when the strongest code-level metric is added as a covariate — prompt complexity is a dimension in its own right.
- Held-out validation on **20 components across 6 unseen repositories** — the two best-performing metrics continue to predict maintenance effort.

Take. The methodological discipline here is notable for the LLM-app-quality genre. Prompt-as-Specification is a useful hook: it invites downstream work on Hoare-logic-ish prompt verification and prompt-driven spec mining.

2.4 · PROGRAM-ANALYSIS-VIA-LLM · Sensitivity/usefulness study of prompt learning for fully-qualified type-name inference  [_____](#)

On Prompt Learning for FQN Inference: Sensitivity and Usefulness Analysis Z. Luo, Q. Huang, Z.

Xing, J. Sun, Q. Lu, J. Lu — ACM TKDD, 2026 — [DOI link](#)

FQN inference (recovering a fully-qualified type name from partial, ambiguous, or unresolved code) is a long-running program-analysis micro-task; prompt-learning has been shown to work well on it, but existing evaluations concentrated on end-to-end accuracy rather than on why it worked. This paper conducts a sensitivity and usefulness analysis of prompt-learning for FQN inference: how the model's outputs and structures respond to prompt variation, and where in the pipeline the win actually comes from.

Take (paywall — abstract-only). Useful primarily as a methodology reference for anyone building prompt-driven type-recovery or symbol-resolution tools; the sensitivity axis is the missing piece in most prompt-based analysis papers. Deep read deferred until the PDF is accessible.

Borderline High-Priority

3.1 · AI-FOR-SE-SLR · Systematic review of AI for software-architecture practice — reference map, not methodology 🙌😊👉

Artificial Intelligence Support for Software Architecture Practice: A Systematic Review and Future Directions A. Bucaioni, M. Weyssow, J. He, Y. Lyu, D. Lo — ACM TOSEM, 2026 — [DOI link](#)

AI is being applied across software engineering, but its explicit role in software *architecture* — a discipline dominated by long-term trade-offs, documentation, and evolution rather than code-level tasks — is still weakly understood. This SLR maps the current landscape and calls out future directions.

Take. Value is in reference-mining rather than methodological novelty. Keep as a citation source when writing anything adjacent (architectural drift, LLM-assisted architecture decision records, etc.); a full deep read is only warranted if you are actively working in the space.

Cross-Paper Synthesis

Two currents dominate today's set.

The first is a coming-of-age moment for **static analysis of LLM-agent software**. AgentFlow (1.1) and HECATE (2.3) reach the same conclusion from opposite directions: prompt-layer semantics are first-class program artefacts and cannot be reasoned about with code-only tooling. AgentFlow lifts prompts, tools, memory, and control policies into a typed dependency graph; HECATE lifts prompts into Hoare-logic specifications. Between them they sketch a “prompt IR” agenda where prompts become typed language objects with their own analysis theory. SFA-Miner (1.2), though rooted in classical API misuse detection, is orthogonally useful: the symbolic-FSA machinery for path-sensitive pattern mining is exactly what an ADG-based taint analyzer would need to reason about conditional prompt-to-tool flows.

The second is the *reliability gap between apparent and actual LLM-tool behavior*. BOUND (2.1) targets output reliability — hallucinated package names look plausible but seed supply-chain attacks. Refploit (2.2) targets execution reliability — agent-generated exploits look successful but sometimes never trigger the vulnerable API. The response pattern is identical and worth internalizing: assume the model is close-but-wrong, use post-hoc localization or differential execution to detect the failure, and surgically correct rather than retrain end-to-end.

Writing & Rationale Insights

BOUND's framing of *package-validity boundary* is a small rhetorical masterstroke: it lets a technique that many audiences distrust (model editing) inherit the intuitive appeal of a decision surface. Refploit's core sentence — “a failed agent trajectory is not entirely useless; it may have already completed some reproduction subtasks while also revealing misleading directions” — is a good template for any paper reframing “failure” as “partial signal.” AgentFlow's ADG deliberately avoids inventing a new IR from scratch and instead decouples framework semantics from graph shape, which is why it can cover five frameworks at once — the right generalization altitude for a paper that hopes to become tooling. HECATE's decision to discard any metric that loses significance under a size covariate is the discipline

the LLM-app-quality genre has been missing; whenever a new metric claims to capture something beyond LoC, this is now the bar.